GlassBox-AutoML Agent

Presented by:

Salim Qadda
Soufiane Ait Lhadj
Salma Ammari
Assia Snissi

Vision & Core Architecture

- **Zero Heavy Dependencies:** Eliminates reliance on bulky frameworks like Scikit-Learn.
- **Math Core:** Powered strictly by Python 3.11+ and NumPy for matrix operations.
- **Modular "Transformer" Design:** Consistent object-oriented API featuring standard fit(), transform(), and predict() methodologies.
- **Highly Optimized:** Utilizes vectorized matrix operations to ensure rapid execution speeds in pure Python.

What is GlassBox?

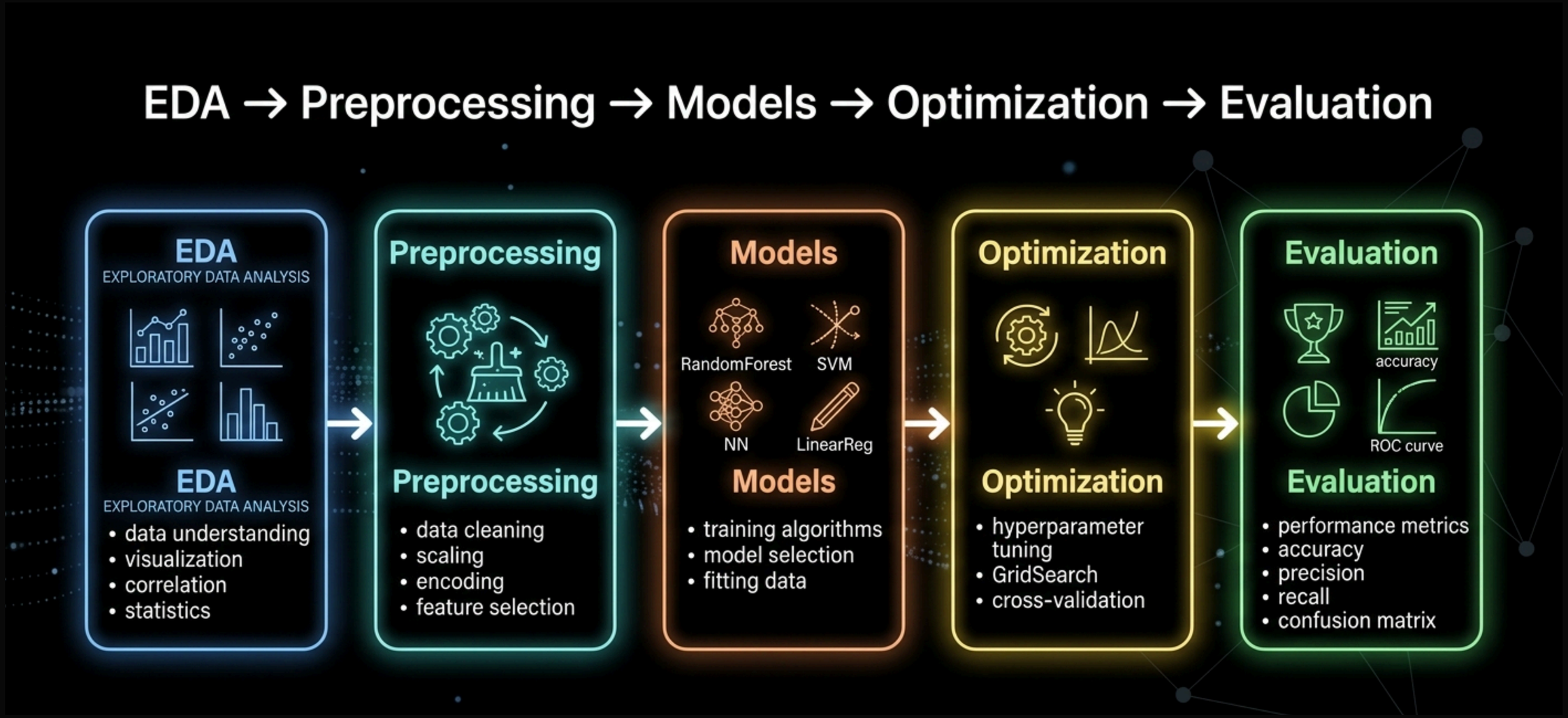
What is GlassBox?

- White-box AutoML
- Built from scratch
- NumPy only

Goals:

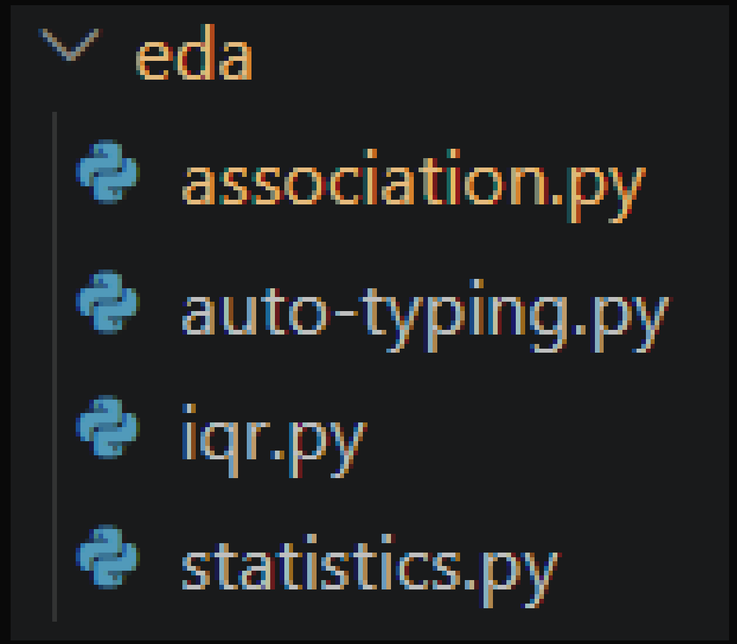
- Explainable
- Modular
- Reproducible

Pipeline


EDA → Preprocessing → Models → Optimization → Evaluation
EDA
EXPLORATORY DATA ANALYSIS
EDA
EXPLORATORY DATA ANALYSIS
• data understanding
• visualization
• correlation
• statistics
Preprocessing
Preprocessing
• data cleaning
• scaling
• encoding
• feature selection
Models
RandomForest
SVM
NN
LinearReg
Models
• training algorithms
• model selection
• fitting data
Optimization
Optimization
• hyperparameter tuning
• GridSearch
• cross-validation
Evaluation
accuracy
ROC curve
Evaluation
• performance metrics
• accuracy
• precision
• recall
• confusion matrix

The Inspector: Data Profiling & Typing

- **Context Gathering:** Non-destructive raw data audits.
- **Statistical Profiling:** Scratch-built implementations for:
  - Central Tendency (Mean, Median, Mode)
  - Distribution characteristics (Standard Deviation, Skewness, Kurtosis)
- **Auto-Typing Engine:** Autonomous logic designed to flawlessly distinguish between Numerical, Categorical, and Boolean features.



The Inspector: Outliers & Relationships

- **Association Mining:** Custom Pearson Correlation Matrix computed from scratch to identify feature collinearity and relationships.

- **Outlier Detection:** Automated Interquartile Range (IQR) implementation.
  - Flags or defensively caps erratic data points falling beyond 1.5×IQR.

```python
def pearson_correlation_matrix(X):
    X = np.asarray(X, dtype=float)

    if X.ndim != 2:
        raise ValueError("pearson_correlation_matrix expects a 2D array")
    if X.shape[0] <= 1:
        raise ValueError("pearson_correlation_matrix requires at least two rows")

    n_features = X.shape[1]
    corr = np.eye(n_features, dtype=float) # Identity matrix In

    for i in range(n_features):
        for j in range(i + 1, n_features):
            value = pearson(X[:, i], X[:, j])
            corr[i, j] = value
            corr[j, i] = value

    return corr
```

```python
def iqr_outliers(x, factor=FACTOR):
    x = np.asarray(x, dtype=float)

    if x.size == 0:
        raise ValueError("iqr_outliers requires at least one value")

    lower, upper = iqr_bounds(x, factor)
    # check if outlier or not
    return (x < lower) | (x > upper)
```

The cleaner: The preprocessor guy

LE — Label Encoder

fit()

- Scans column for unique categories
- Sorts alphabetically
- Assigns each category an integer index

transform()

- Replaces each value with its integer code
- Unseen categories → assigned **-1**

example

`cat → 0 dog → 1 ? → -1`

Output: 1D integer array

OHE — One-Hot Encoder

fit()

- Scans column for unique categories
- Assigns each category its own column index
- Stores mapping dict

transform()

- Returns binary matrix (0s and 1s)
- Fires the matching column
- Unseen → **unknown col**

example

`cat→[1,0,0] dog→[0,1,0]`

Output: 2D binary matrix

SI — Simple Imputer

fit()

- Detects column type (numeric vs categorical)
- Numeric → stores **mean** or **median**
- Categorical → stores **mode**

transform()

- Scans for NaN / None
- Fills with stored value
- Works on a copy (original untouched)

example

`NaN → mean(col) None → mode`

Output: filled matrix

MMS — MinMax Scaler

fit()

- Computes **min** and **max** per column
- Stores them for later
- Only on training data

transform()

- Applies formula per col

$$(x - \min) / (\max - \min)$$

- Output always in **[0, 1]**
- Constant col → range=1

sensitive to outliers

`outlier pulls min/max`

Output: scaled [0,1] matrix

SS — Standard Scaler

fit()

- Computes **mean** and **std** per column
- Stores them for later
- Only on training data

transform()

- Applies formula per col

$$(x - \mu) / \sigma$$

- Output centered at **0**
- Constant col → std=1

sensitive to outliers

`outlier shifts mean/std`

Output: z-score matrix

fit() learns from training data only — transform() can be applied to any split (train or test) — this prevents data leakage

The Models: White-Box Implementations

- Every algorithm in this suite is written natively using strictly NumPy.
- Ensures absolute transparency: users can inspect exactly how weights are updated or trees are split.
- Supports both Classification and Regression tasks across the board.

```python
class BaseModel(ABC):
    """
    Abstract base class for all models in the Algorithm Zoo.
    Every model must implement fit() and predict().
    """

    @abstractmethod
    def fit(self, X, y):
        pass

    @abstractmethod
    def predict(self, X):
        pass
    def score(self, X, y):
```

Linear Regression

fit() — solver='gd'

- Initializes **w = 0** (one weight per feature + bias)
- Iterates up to **n_epochs**, updating w each step
- Stops early if cost improvement < **tol**
- Stores learned weights in **w_** and cost history in **costs_**

① Cost Function (MSE)

$$J(w) = (1/2m) \cdot \|Xw - y\|^2$$

② Gradient of J

$$\nabla J(w) = (1/m) \cdot X^T(Xw - y)$$

③ Algorithm

Initialize:
w ← zeros(n_features + 1)

for epoch = 1 … n_epochs:

- compute cost J(w)
- **if** |J_prev − J| < tol → **stop**
- update:
  $w \leftarrow w - lr \cdot \nabla J(w)$

Intuition:
Each step nudges w in the direction that reduces the cost the most (steepest descent)

predict()

- Prepends a column of ones to X (bias)
- Returns **ŷ = [1|X] · w** (dot product)
- Same for both solvers — only *fit()* differs

fit() — solver='normal'

- No iterations — computes **w** in a single matrix op
- No hyperparameters needed (lr, epochs, tol unused)
- Uses **pseudoinverse** to handle collinear features
- Stores result directly in **w_**

① Set gradient to zero and solve for w

$$\nabla J(w) = 0 \rightarrow X^TXw = X^Ty$$

② Closed-form solution

$$w = (X^TX)^{-1} X^Ty$$

③ In code — using pseudoinverse

```
w = pinv(X) @ y
```

Why pseudoinverse instead of regular inverse?
X^TX can be singular (non-invertible) when features are collinear. pinv() handles this gracefully — no crash, still finds best solution.

predict()

- Prepends a column of ones to X (bias)
- Returns **ŷ = [1|X] · w** (dot product)
- Identical to GD solver — only *fit()* differs

Why use Closed Form over Gradient Descent?

Advantages of Normal Equation

No hyperparameters to tune
No need to choose lr, n_epochs, or tol

Exact solution in one step
No iteration — no risk of slow or failed convergence

Always finds the global minimum
MSE is convex — closed form is guaranteed optimal

No learning rate sensitivity
GD can diverge or crawl with a wrong lr value

When Gradient Descent is preferred

Large datasets (many rows)
pinv(X) is $O(n^3)$ — very slow for large n

Many features (wide matrix)
Matrix inversion cost grows with $n_features^2$

Online / streaming learning
GD can update incrementally — normal eq cannot

Bottom line

Small dataset → **Normal Equation** (fast, exact, no tuning)
Large dataset → **Gradient Descent** (scalable, iterative)

Logistic Regression

fit() — One-vs-Rest strategy

- Prepends bias column to X
- Learns unique class labels → stores in **classes_**
- Trains **K binary classifiers** — one per class
- Stores weight vectors in **weights_** shape (K, n+1)

① Cost (Binary Cross-Entropy)

$$J(w) = -(1/m) \cdot \Sigma \left[y \cdot \log(p) + (1-y) \cdot \log(1-p) \right]$$

② Sigmoid (maps Xw → probability)

$$\sigma(z) = 1 / (1 + e^{-z})$$

③ Gradient of J

$$\nabla J(w) = (1/m) \cdot X^T(\sigma(Xw) - y)$$

Why is the gradient so clean?
$\sigma'(z) = \sigma(1-\sigma)$ cancels with the log derivative — leaving just $\sigma(Xw) - y$

④ Algorithm (per binary classifier)

Initialize:
w ← zeros(n_features + 1)

for epoch = 1 … n_epochs:

- compute cost J(w)
- **if** |J_prev − J| < tol → **stop**
- update:
  w ← w − lr · ∇J(w)

Repeated K times — once per class

predict_proba() + predict()

predict_proba()

- Prepends bias column to X
- Runs each of the K weight vectors through sigmoid:

$$p_k = \sigma(X \cdot w_k) \text{ for each } k$$

- Assembles probability matrix of shape **(m, K)**
- Normalizes rows so they sum to 1:

$$p_k \leftarrow p_k / \Sigma\, p_k$$

Why normalize?
Each binary classifier is independent — their raw scores can sum to anything, not just 1

predict()

- Calls **predict_proba()** to get (m, K) matrix
- Takes **argmax** along axis=1 — highest prob wins
- Maps index back to original class label via classes_

$$\hat{y} = classes_[\ argmax(probs)\ ]$$

Note on threshold
threshold=0.5 is defined but unused — predict() uses argmax, which is correct for multi-class

Numerical stability

- z clipped to **[-500, 500]** — prevents overflow in e^{-z}
- p clipped to **[1e-15, 1-1e-15]** — prevents log(0)

Why One-vs-Rest (OvR)?

What is OvR?

- K classes → train **K binary classifiers**
- Each asks: ***"is this my class or not?"***
- At prediction: class with highest score wins

Why it is a good choice here

Simple to implement
Reuses the same binary GD logic K times — no new math

Scales linearly with classes
K classes → K classifiers (OvO needs K(K-1)/2)

Works well in practice
Good performance when classes are well separated

Known weaknesses

Class imbalance per binary problem
1 positive class vs K-1 negatives — can bias training

Probabilities need normalization
Raw OvR scores don't sum to 1 — a principled solution would use Softmax instead

OvR vs Softmax

OvR — simpler, modular, easier to build
Softmax — one classifier, probs naturally sum to 1 — what sklearn uses

Decision Trees

- **How It Works:** Recursively divides the dataset to find the most optimal patterns, built entirely in pure NumPy.

- **Splitting Logic:**
  - **Classification:** Uses **Gini Impurity** to maximize accuracy.
  - **Regression:** Uses **Mean Squared Error (MSE)** to minimize variance.

- **Built-in Guardrails:** Utilizes max_depth and min_samples_split constraints to prevent the model from memorizing training data.

```
class Node:
    def __init__(self,
                 feature=None,
                 threshold=None,
                 left=None,
                 right=None,
                 prediction=None,
                 samples_count=0,
                 impurity=0.0):
```

```
class DecisionTree(BaseModel):
    def __init__(self, max_depth=10,
                 min_samples_split=2,
                 n_features=None,
                 task='classification'):
```

Random Forest

- **Ensemble of Decision Trees**

  Uses B.A.G.ging — each tree trains on a bootstrap sample (random rows with replacement, ~33% left out as Out-of-Bag data).

- **Feature Randomness at every split**

  Only √n_features considered per node. Forces diverse, expert trees instead of identical clones.

- **Variance = σ^2 / N**

  Averaging N trees reduces variance by factor N while preserving low bias from individual trees.

- **Classification & Regression**

  Majority vote for classification. Mean of predictions for regression tasks.

```python
def fit(self, X, y):
    self.trees = []
    for _ in range(self.n_trees):
        # 1. Create a Bootstrap sample (Random rows with replacement)
        X_sample, y_sample = self._bootstrap_samples(X, y)

        # 2. Initialize your existing DecisionTree
        tree = DecisionTree(
            max_depth=self.max_depth,
            min_samples_split=self.min_samples_split,
            n_features=self.n_features,
            task=self.task
        )

        # 3. Train the tree on the random sample
        tree.fit(X_sample, y_sample)
        self.trees.append(tree)
    return self
```

```python
def _bootstrap_samples(self, X, y): # X features matrix with rows filled
    n_samples = X.shape[0] # num of rows
    # Pick random indices (with replacement)
    #index positions of the data I want to grab.
    indices = np.random.choice(n_samples, n_samples, replace=True)
    return X[indices], y[indices]
```

Gaussian Naïve Bayes

- Bayes part : we use bayes theorem

$$P(C|x) = \frac{P(x|C) \cdot P(C)}{P(x)}$$

- Naive part : Assumes feature independence this simplifies calculations by assuming features don't depend on each other

$$P(x|C) = \prod_{i=1}^{n} P(x_i|C)$$

- Gaussian part :

  For continuous features :

$$P(x_i \mid C) = \frac{1}{\sqrt{2\pi\sigma_{C,i}^2}} \exp\left(-\frac{(x_i - \mu_{C,i})^2}{2\sigma_{C,i}^2}\right)$$

For Categorical features :

$$P(x_i = v \mid C) = \frac{\text{count}(x_i = v \text{ in class } C)}{\text{total samples in class } C}$$

K-Nearest Neighbors (KNN)

```python
import numpy as np
from models.base_model import BaseModel

class KNearestNeighbors(BaseModel):
    """K-Nearest Neighbors classifier/regressor with optimizations."""

    def __init__(self, k=5, distance_metric="euclidean", task="classification
        self.k = k
        self.distance_metric = distance_metric
        self.task = task
        # Pre-compute distance function for faster access
        self._distance_fn = self._get_distance_function()

    def fit(self, X, y):
        """Fit the model with training data."""
        self.X_train = np.asarray(X, dtype=np.float32)
        self.y_train = np.asarray(y)
        # Pre-cache training set size for faster loops
        self._train_size = len(self.X_train)

    def _get_distance_function(self):
        """Return the appropriate distance function as a closure."""
        if self.distance_metric == "euclidean":
            return lambda x1, x2: np.sqrt(np.sum((x1 - x2) ** 2))
        elif self.distance_metric == "manhattan":
            return lambda x1, x2: np.sum(np.abs(x1 - x2))
        else:
            raise ValueError("Unsupported distance metric")
```

```python
def _distance(self, x1, x2):
    """Calculate distance between two points (maintains API compatibility)."""
    return self._distance_fn(x1, x2)

def _predict_single(self, x):
    """Predict for a single sample using optimized numpy operations."""
    # Vectorized distance computation instead of loop
    distances = np.sqrt(np.sum((self.X_train - x) ** 2, axis=1)) \
        if self.distance_metric == "euclidean" \
        else np.sum(np.abs(self.X_train - x), axis=1)

    # Use argpartition for O(n) k-selection instead of full sort O(n log n)
    k_indices = np.argpartition(distances, min(self.k, len(distances) - 1))[:self.k
    neighbor_values = self.y_train[k_indices]

    if self.task == "classification":
        labels, counts = np.unique(neighbor_values, return_counts=True)
        return labels[np.argmax(counts)]
    elif self.task == "regression":
        return np.mean(neighbor_values)
    else:
        raise ValueError("Unsupported task type")
  def predict(self, X):
      """Predict for multiple samples."""
      X = np.asarray(X, dtype=np.float32)
      predictions = np.empty(len(X), dtype=self.y_train.dtype)

      for i, x in enumerate(X):
          predictions[i] = self._predict_single(x)

      return predictions
```

The orchestrator

- Automated hyperparameter tuning via search strategies and cross-validation

- **Components:**

  - **GridSearch:** Exhaustive iteration. Guaranteed optimal.

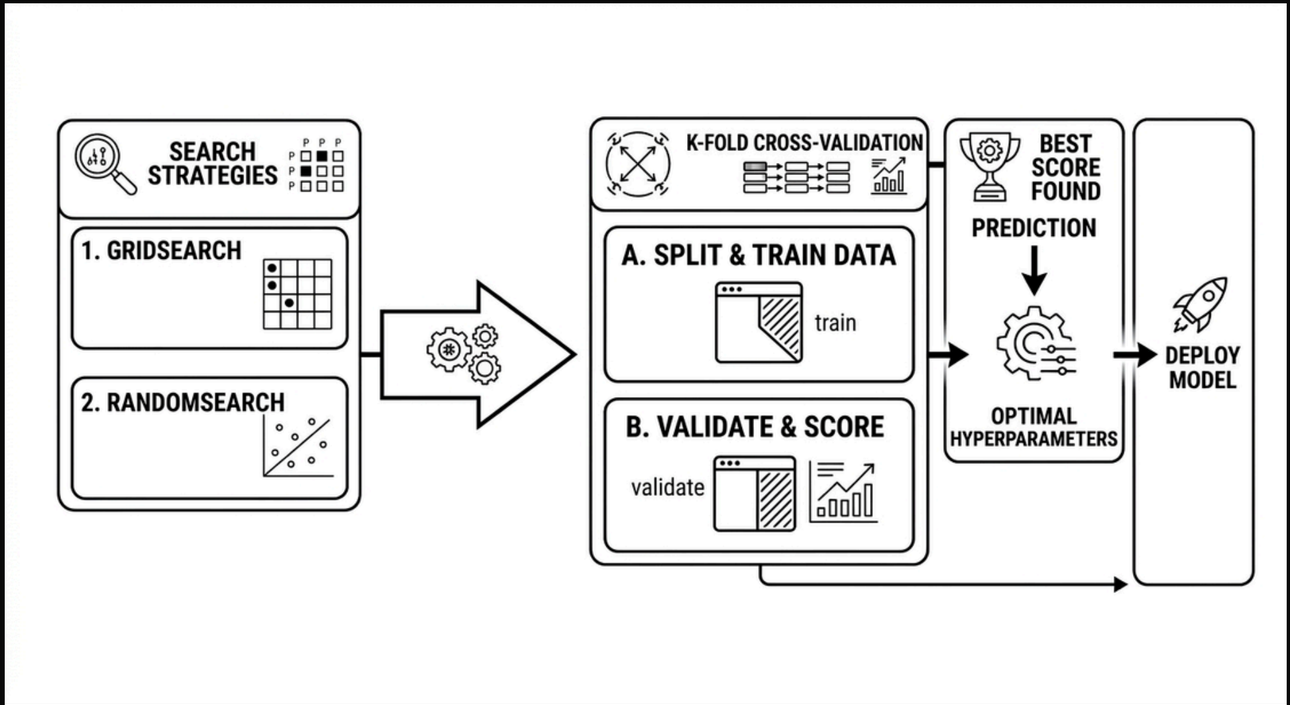
  - **RandomSearch:** Stochastic sampling. Scales to large spaces.

  - **K-Fold CV:** Splits data k ways. Trains k-1, validates 1. Averages results.

The orchestrator

- **Workflow:** GridSearch/RandomSearch → For each combo: K-Fold CV → Track best score → Return optimal hyperparameters

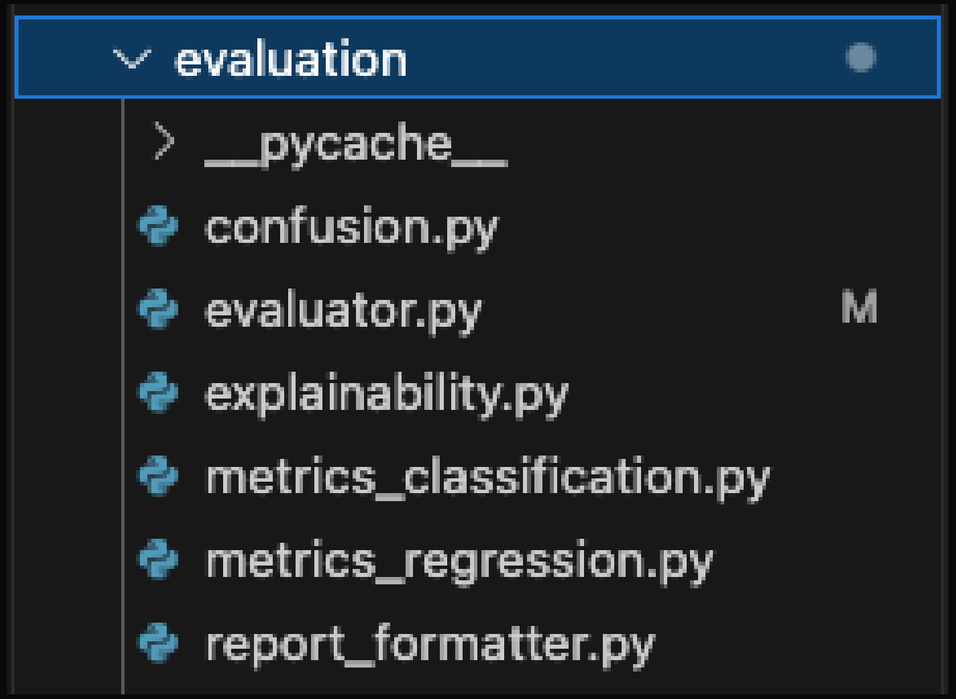
- **Why it matters:** No manual tuning. Transparent, auditable process



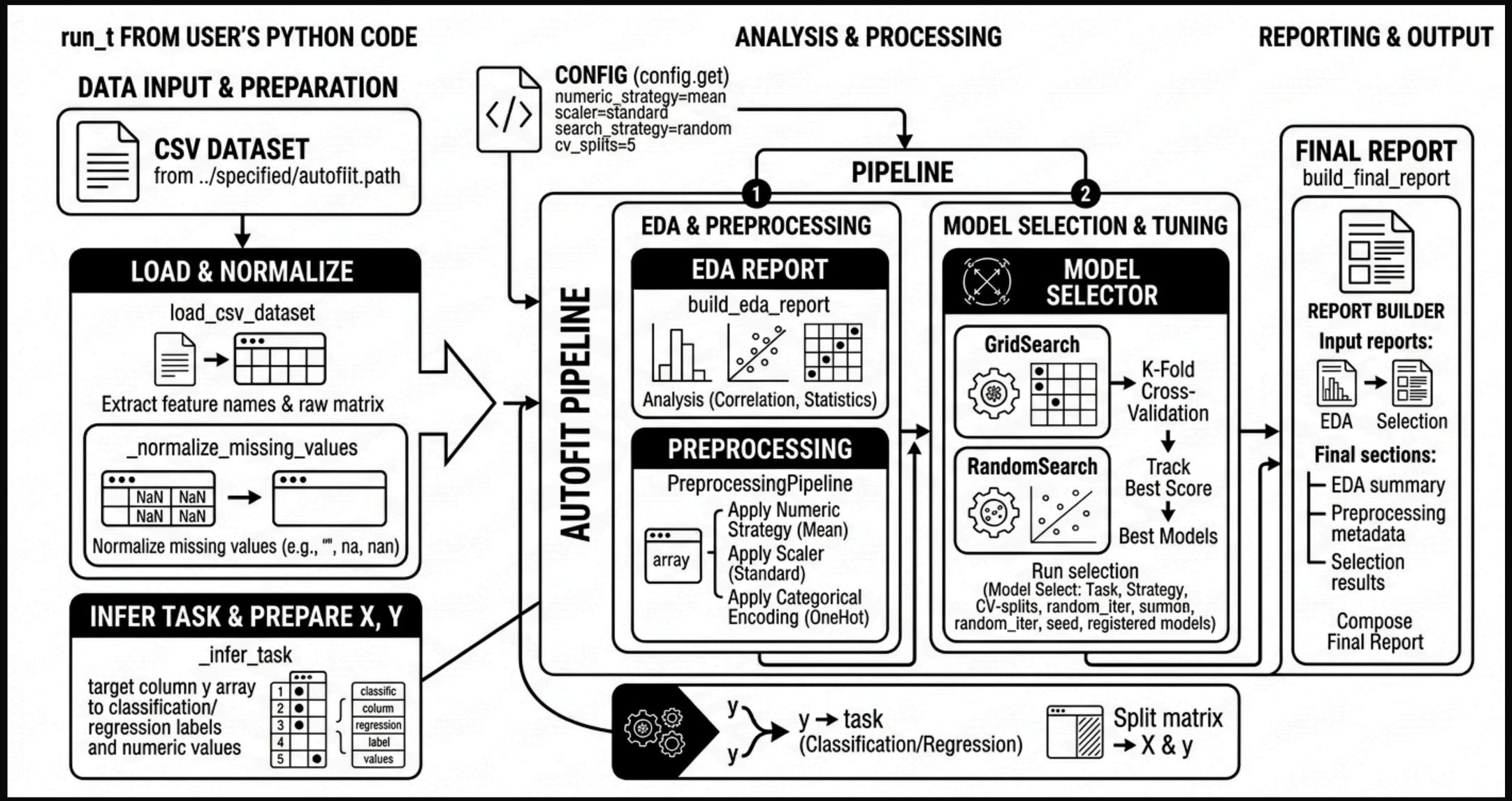
Evaluation

- **Comprehensive Metric Suites** : Provides specialized performance tracking for both Classification (Precision, Recall, F1-Score) and Regression (MSE, MAE, R^2), ensuring the model’s accuracy is validated against industry-standard benchmarks

- **Diagnostic & Visual Analysis** : Features automated Confusion Matrix generation and Explainability tools (such as Feature Importance) to move beyond raw numbers and understand "why" the model is making specific decisions.



AutoFit: The Brain of GlassBox-AutoML



AutoFit → MCP Server →IronClaw

Configuration of IronClaw

```
━━━
```

What worked?

Titanic Dataset (891 rows)

```
[› run autofit tool for this dataset : /Users/mac/Desktop/new_glassbox/GlassBox-AutoML-Agent/data/titanic.csv with target is survived ]
  ● autofit_run_autofit_tool
    "{\n \"status\": \"success\",\n \"input\": {\n \"csv_path\": \"/Users/mac/Desktop/new_glassbox/GlassBox-AutoML-Agent/data/titanic.csv\",\n \"target_column\": \"survived
\",\n \"task\": \"classification...
────────────────────────────────────────────────────────────────────────────────
The autofit tool successfully ran on the titanic dataset. Here's a summary of the findings:

Dataset Summary:
•   Rows: 891
•   Columns: 13 (after processing)
•   Target Column: 'survived'
•   Task: Classification

Exploratory Data Analysis (EDA) Summary:
•   Numerical Features: pclass, age, sibsp, parch, fare
•   Categorical Features: sex, embarked, class, who, adult_male, deck, embark_town, alone
•   Missing Values:
  *   'age': 177 missing
  *   'embarked': 2 missing
  *   'embark_town': 2 missing
  *   'deck': 688 missing (significant number)
•   Correlations: Notable negative correlation between 'pclass' and 'fare'.

Preprocessing Summary:
•   Numerical Strategy: Mean imputation
•   Scaler: Standard scaling
•   Categorical Encoding: One-hot encoding

Search Summary:
•   Strategy: Grid search
•   Cross-validation splits: 5
•   Model Tried: Logistic Regression
•   Best Parameters for Logistic Regression: lr: 0.01, n_epochs: 800, tol: 1e-6

Best Model:
•   Model: Logistic Regression
•   Cross-validation Score: 0.8249

Explainability:
The most influential features for predicting survival were:
•   adult_male__False (likely indicating being female)
•   pclass (passenger class)
•   who__man (likely indicating being a man)
•   sex__female
```

What worked?

Tesla Dataset (255 rows)

```
[› run autofit tool for this dataset : /Users/mac/Desktop/new_glassbox/GlassBox-AutoML-Agent/data/teslay.csv , target column = price ]
  ● autofit_run_autofit_tool
    "{\n \"status\": \"success\",\n \"input\": {\n \"csv_path\": \"/Users/mac/Desktop/new_glassbox/GlassBox-AutoML-Agent/data/teslay.csv\",\n \"target_column\": \"price\",\
n \"task\": \"regression\"\n },\...
────────────────────────────────────────────────────────────────────────────────
The autofit tool successfully ran on the /Users/mac/Desktop/new_glassbox/GlassBox-AutoML-Agent/data/teslay.csv dataset
with 'price' as the target column.

Here's a summary of the results:

•   Dataset: 255 rows, 4 columns.
•   Features: 'year', 'km', and 'color' (which was one-hot encoded into 12 features).
•   Target: 'price' (regression task).
•   Best Model: k-Nearest Neighbors (KNN) with k=7 and Euclidean distance.
•   Performance: The KNN model achieved a cross-validation Mean Squared Error (MSE) of approximately 1.73e10 and an
  R-squared of 0.438 on the training data.

Would you like me to elaborate on any of these results, or perhaps try another analysis?

›
```

What Didn’t

Timeout issue

```
[› run autofit_run_autofit_tool , csv_path =  /Users/mac/Desktop/new_glassbox/GlassBox-AutoML-Agent/data/titanic.csv ,  target_column = survived ]
  × autofit_run_autofit_tool (failed)
────────────────────────────────────────────────────────────────────────────────
I encountered an issue while running the autofit_run_autofit_tool. The tool timed out after 30 seconds. Would you like
me to try again, or perhaps adjust the parameters if possible?

›
```

Future work

Messaging Integration

Expose Ironclaw via **WhatsApp** or **Telegram** so users can trigger the full ML pipeline from a simple chat message — no interface needed

Impact
Makes ML accessible to anyone with a phone

Performance & Speed

Optimize models to handle **large datasets** without hitting Ironclaw's request timeout

Explore mini-batch GD, async execution, or offloading heavy computation

Impact
Makes autofit reliable on real-world sized data

Dynamic LLM Pipeline

Let the LLM decide on **preprocessing steps** and **model selection** dynamically instead of a fixed pipeline — adapting to each dataset's specific characteristics

Impact
Smarter, more flexible autofit for any use case

More Data Sources

Support more input formats beyond local files — such as

URLs · Google Sheets
SQL databases

User just pastes a link or connection string — autofit handles the rest

Impact
No preprocessing needed before talking to Ironclaw

Thank you !